27.2 节点注册与发现

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~260k tokens，输出 ~8k tokens（本节）

上一节介绍了节点的概念和命令分类。本节深入 Gateway 侧的实现：节点如何注册到 Gateway、命令如何被路由和调用、事件如何在节点与会话之间流转、以及订阅系统如何实现实时消息推送。

27.2.1 节点注册表（`src/gateway/node-registry.ts`）

双索引结构

`NodeRegistry` 是 Gateway 中管理所有在线节点的核心类。它使用两个 Map 实现**双向索引**：

```
// src/gateway/node-registry.ts

export class NodeRegistry {
  private nodesById   = new Map<string, NodeSession>();  // nodeId → NodeSession
  private nodesByConn = new Map<string, string>();        // connId → nodeId
  private pendingInvokes = new Map<string, PendingInvoke>(); // requestId → 挂起的调用
  // ...
}
```

- `nodesById`：通过节点 ID 查找节点会话——Agent 发起 `node.invoke` 时用此索引。
- `nodesByConn`：通过 WebSocket 连接 ID 查找节点 ID——连接断开时用此反查。

> **衍生解释：双向索引（Bidirectional Index）**
>
> 在数据库领域，索引用于加速查询。单向索引只支持一个方向的查找，例如 `Map<K, V>` 只能通过键查值。双向索引同时维护两个方向的映射，使得两个方向的查找都是 O(1)。代价是写入时需要同时更新两个 Map，并且要保证它们的一致性。这是一个经典的**空间换时间**策略。

注册与注销

节点通过 WebSocket 连接到 Gateway 后， `register` 方法将其信息提取并存入注册表：

```
// src/gateway/node-registry.ts

register(client: GatewayWsClient, opts: { remoteIp?: string }) {
  const connect = client.connect;
  const nodeId = connect.device?.id ?? connect.client.id;
  const caps = Array.isArray(connect.caps) ? connect.caps : [];
  const commands = Array.isArray(connect.commands) ? connect.commands : [];
  const permissions = typeof connect.permissions === "object"
    ? connect.permissions : undefined;
  const pathEnv = typeof connect.pathEnv === "string"
    ? connect.pathEnv : undefined;

  const session: NodeSession = {
    nodeId,
    connId: client.connId,
    client,
    displayName: connect.client.displayName,
    platform: connect.client.platform,
    // ... 更多字段
    caps,
    commands,
    permissions,
    pathEnv,
    connectedAtMs: Date.now(),
  };

  this.nodesById.set(nodeId, session);
  this.nodesByConn.set(client.connId, nodeId);
  return session;
}
```

`nodeId` 的选取策略值得关注：优先使用 `device.id` （设备持久标识），如果不存在则回退到 `client.id` 。这确保了同一台设备重新连接时 `nodeId` 不变。

注销时需要额外处理**挂起的调用**：

```
// src/gateway/node-registry.ts

unregister(connId: string): string | null {
  const nodeId = this.nodesByConn.get(connId);
  if (!nodeId) return null;

  this.nodesByConn.delete(connId);
  this.nodesById.delete(nodeId);

  // 清理该节点上所有挂起的调用
  for (const [id, pending] of this.pendingInvokes.entries()) {
    if (pending.nodeId !== nodeId) continue;
    clearTimeout(pending.timer);
    pending.reject(new Error(`node disconnected (${pending.command})`));
    this.pendingInvokes.delete(id);
  }
  return nodeId;
}
```

当节点断开连接时，所有等待该节点返回结果的 Promise 都会被 `reject`，通知调用方节点已离线。这样就不会有人傻等一个永远不会到来的结果了。

命令调用（invoke）

`invoke` 方法是节点系统的核心——它将命令发送给目标节点，并返回一个 Promise 等待结果：

```typescript
// src/gateway/node-registry.ts

async invoke(params: {
  nodeId: string;
  command: string;
  params?: unknown;
  timeoutMs?: number;
  idempotencyKey?: string;
}): Promise<NodeInvokeResult> {
  const node = this.nodesById.get(params.nodeId);
  if (!node) {
    return { ok: false, error: { code: "NOT_CONNECTED", message: "node not connected" }
  }

  const requestId = randomUUID();
  const payload = {
    id: requestId,
    nodeId: params.nodeId,
    command: params.command,
    paramsJSON: params.params !== undefined ? JSON.stringify(params.params) : null,
    timeoutMs: params.timeoutMs,
    idempotencyKey: params.idempotencyKey,
  };

  // 通过 WebSocket 发送给节点
  const ok = this.sendEventToSession(node, "node.invoke.request", payload);
  if (!ok) {
    return { ok: false, error: { code: "UNAVAILABLE", message: "failed to send" } };
  }

  // 等待结果（带超时）
  const timeoutMs = typeof params.timeoutMs === "number" ? params.timeoutMs : 30_000;
  return await new Promise<NodeInvokeResult>((resolve, reject) => {
    const timer = setTimeout(() => {
      this.pendingInvokes.delete(requestId);
      resolve({ ok: false, error: { code: "TIMEOUT", message: "node invoke timed out" }
    }, timeoutMs);
    this.pendingInvokes.set(requestId, {
      nodeId: params.nodeId,
      command: params.command,
      resolve, reject, timer,
    });
```

```
  });
}
```

这段代码展示了一个经典的**请求-响应匹配**模式：

1. 生成唯一的 `requestId` （UUID）。
2. 将请求发送给节点，同时将 `{ resolve, reject, timer }` 保存到 `pendingInvokes` 。
3. 节点执行完毕后调用 `handleInvokeResult` ，通过 `requestId` 找到对应的 `resolve` 并完成 Promise。
4. 如果超时， `timer` 会自动 `resolve` 一个超时错误（注意不是 `reject` ——这是有意的设计，使得超时是一种"正常"的失败结果）。

> **衍生解释：幂等键（Idempotency Key）**
>
> `idempotencyKey` 参数用于保证**幂等性**——即相同的请求多次执行产生相同的效果。在分布式系统中，网络不可靠可能导致请求被重复发送。接收方通过检查 idempotency key 来判断是否已经处理过该请求，如果是则直接返回之前的结果。这在金融交易等场景中尤为重要。

结果匹配

当节点返回结果时， `handleInvokeResult` 方法将其与挂起的请求匹配：

```
// src/gateway/node-registry.ts

handleInvokeResult(params: {
  id: string;       // requestId
  nodeId: string;
  ok: boolean;
  payload?: unknown;
  payloadJSON?: string | null;
  error?: { code?: string; message?: string } | null;
}): boolean {
  const pending = this.pendingInvokes.get(params.id);
  if (!pending) return false;              // 可能已超时
  if (pending.nodeId !== params.nodeId) return false; // 安全校验

  clearTimeout(pending.timer);
  this.pendingInvokes.delete(params.id);
  pending.resolve({
    ok: params.ok,
    payload: params.payload,
    payloadJSON: params.payloadJSON ?? null,
    error: params.error ?? null,
  });
  return true;
}
```

如果 `pending` 不存在（超时后到达的迟到结果），方法返回 `false`，Gateway 会将其标记为 `ignored` ——这是**最终一致性**的一种体现。

27.2.2 节点事件系统（`src/gateway/server-node-events.ts`）

事件 vs 命令调用

节点系统有两种通信模式：

模式	方向	语义	等待响应
命令调用（invoke）	Gateway → Node → Gateway	请求-响应	是（Promise）

模式	方向	语义	等待响应
事件（event）	Node → Gateway	单向通知	否

事件由节点主动发送给 Gateway，用于报告状态变更或传递数据。 `handleNodeEvent` 是事件处理的分发器：

```
// src/gateway/server-node-events.ts

export const handleNodeEvent = async (
  ctx: NodeEventContext,
  nodeId: string,
  evt: NodeEvent,
) => {
  switch (evt.event) {
    case "voice.transcript":   // 语音转文字结果
      // → 创建会话，触发 Agent 处理
    case "agent.request":      // Agent 深链请求
      // → 解析参数，执行 Agent 命令
    case "chat.subscribe":     // 订阅会话消息
      // → 注册节点到订阅系统
    case "chat.unsubscribe":   // 取消订阅
      // → 从订阅系统移除
    case "exec.started":       // 命令执行开始
    case "exec.finished":      // 命令执行完成
    case "exec.denied":        // 命令执行被拒绝
      // → 注入系统事件，触发心跳
  }
};
```

语音转写事件（voice.transcript）

当 iOS/Android 节点完成语音转写后，将文本发送给 Gateway：

```
// src/gateway/server-node-events.ts（简化）

case "voice.transcript": {
  const payload = JSON.parse(evt.payloadJSON);
  const text = payload.text?.trim();
  if (!text || text.length > 20_000) return;

  // 解析目标会话
  const sessionKey = payload.sessionKey || mainKey;
  const { storePath, entry, canonicalKey } = loadSessionEntry(sessionKey);
  const sessionId = entry?.sessionId ?? randomUUID();

  // 注册 chatRun，使 UI 能刷新
  ctx.addChatRun(sessionId, {
    sessionKey,
    clientRunId: `voice-${randomUUID()}`,
  });

  // 触发 Agent 处理
  void agentCommand({
    message: text,
    sessionId,
    sessionKey,
    thinking: "low",
    deliver: false,
    messageChannel: "node",
  }, defaultRuntime, ctx.deps);
  return;
}
```

这段代码展示了一个完整的**语音交互闭环**：用户在手机上说话 → 手机端完成语音识别 → 识别结果通过 WebSocket 事件发送到 Gateway → Gateway 创建 Agent 会话处理文本 → Agent 回复通过订阅系统推送回手机。

Agent 深链请求（agent.request）

`agent.request` 是一种更通用的事件，允许节点以**深链（Deep Link）** 的方式触发 Agent 执行：

```
// src/gateway/server-node-events.ts（简化）

case "agent.request": {
  const link = JSON.parse(evt.payloadJSON);
  const message = link.message?.trim();
  if (!message || message.length > 20_000) return;

  const channel = normalizeChannelId(link.channel) ?? undefined;
  const to = link.to?.trim() || undefined;
  const deliver = Boolean(link.deliver) && Boolean(channel);
  const sessionKey = link.sessionKey || `node-${nodeId}`;

  void agentCommand({
    message,
    sessionId,
    sessionKey,
    thinking: link.thinking,
    deliver,
    to,
    channel,
    timeout: link.timeoutSeconds?.toString(),
    messageChannel: "node",
  }, defaultRuntime, ctx.deps);
  return;
}
```

与 `voice.transcript` 不同， `agent.request` 支持更多参数：指定目标通道（ `channel` ）、收件人（ `to` ）、是否投递（ `deliver` ）等。这使得节点可以触发复杂的 Agent 流程，例如"识别图片内容并通过 Telegram 发送给特定联系人"。

执行事件（exec.*）

Node Host 在执行系统命令时会发送三类事件：

```
// src/gateway/server-node-events.ts（简化）

case "exec.started":
case "exec.finished":
case "exec.denied": {
  const sessionKey = payload.sessionKey || `node-${nodeId}`;
  const runId = payload.runId;
  const command = payload.command;

  let text = "";
  if (evt.event === "exec.started") {
    text = `Exec started (node=${nodeId}): ${command}`;
  } else if (evt.event === "exec.finished") {
    const exitLabel = payload.timedOut ? "timeout" : `code ${payload.exitCode}`;
    text = `Exec finished (node=${nodeId}, ${exitLabel})`;
    if (payload.output) text += `\n${payload.output}`;
  } else {
    text = `Exec denied (node=${nodeId}, ${payload.reason}): ${command}`;
  }

  enqueueSystemEvent(text, { sessionKey, contextKey: `exec:${runId}` });
  requestHeartbeatNow({ reason: "exec-event" });
  return;
}
```

这些事件被转化为**系统事件**注入到会话上下文中，使得 Agent 可以感知到远程命令的执行进度和结果。 `requestHeartbeatNow` 触发心跳，确保 Agent 及时处理这些事件。

27.2.3 节点订阅（`src/gateway/server-node-subscriptions.ts`）

为什么需要订阅

考虑这样一个场景：用户在 iPhone 上通过 OpenClaw iOS App 对话。Agent 的回复需要实时推送到这台 iPhone。但 Gateway 怎么知道"这条回复应该推给哪个节点"？

答案是**订阅机制**。节点通过 `chat.subscribe` 事件告诉 Gateway："我对 sessionKey=xxx 的会话感兴趣，请把相关消息推给我。"

双向映射结构

`createNodeSubscriptionManager` 使用两个 Map 维护节点与会话之间的**多对多**订阅关系：

```
// src/gateway/server-node-subscriptions.ts

export function createNodeSubscriptionManager(): NodeSubscriptionManager {
  // nodeId → Set<sessionKey>  ——一个节点可以订阅多个会话
  const nodeSubscriptions = new Map<string, Set<string>>();
  // sessionKey → Set<nodeId>  ——一个会话可以被多个节点订阅
  const sessionSubscribers = new Map<string, Set<string>>();
  // ...
}
```

> **衍生解释：多对多关系（Many-to-Many Relationship）**
>
> 在关系型数据库中，多对多关系通常通过中间表（Junction Table）实现。这里使用两个 Map 做双向索引，本质是同一思想的内存实现。 `nodeSubscriptions` 相当于"节点 → 会话"方向的索引， `sessionSubscribers` 相当于"会话 → 节点"方向的索引。两个 Map 必须保持同步更新。

订阅与退订

```
// src/gateway/server-node-subscriptions.ts

const subscribe = (nodeId: string, sessionKey: string) => {
  // 更新 nodeSubscriptions: nodeId → sessionKeys
  let nodeSet = nodeSubscriptions.get(nodeId);
  if (!nodeSet) {
    nodeSet = new Set<string>();
    nodeSubscriptions.set(nodeId, nodeSet);
  }
  if (nodeSet.has(sessionKey)) return; // 幂等——重复订阅无副作用
  nodeSet.add(sessionKey);

  // 更新 sessionSubscribers: sessionKey → nodeIds
  let sessionSet = sessionSubscribers.get(sessionKey);
  if (!sessionSet) {
    sessionSet = new Set<string>();
    sessionSubscribers.set(sessionKey, sessionSet);
  }
  sessionSet.add(nodeId);
};
```

退订时同步清理两个 Map，并在 Set 为空时删除整个条目以避免内存泄漏：

```
const unsubscribe = (nodeId: string, sessionKey: string) => {
  const nodeSet = nodeSubscriptions.get(nodeId);
  nodeSet?.delete(sessionKey);
  if (nodeSet?.size === 0) nodeSubscriptions.delete(nodeId);

  const sessionSet = sessionSubscribers.get(sessionKey);
  sessionSet?.delete(nodeId);
  if (sessionSet?.size === 0) sessionSubscribers.delete(sessionKey);
};
```

当节点断开连接时， `unsubscribeAll` 会一次性清除该节点的所有订阅：

```
const unsubscribeAll = (nodeId: string) => {
  const nodeSet = nodeSubscriptions.get(nodeId);
  if (!nodeSet) return;
  for (const sessionKey of nodeSet) {
    const sessionSet = sessionSubscribers.get(sessionKey);
    sessionSet?.delete(nodeId);
    if (sessionSet?.size === 0) sessionSubscribers.delete(sessionKey);
  }
  nodeSubscriptions.delete(nodeId);
};
```

消息分发

订阅管理器提供三种分发方式：

```
// 1. 发送给订阅了特定会话的所有节点
sendToSession(sessionKey, event, payload, sendEvent) {
  const subs = sessionSubscribers.get(sessionKey);
  if (!subs) return;
  const payloadJSON = JSON.stringify(payload);
  for (const nodeId of subs) {
    sendEvent({ nodeId, event, payloadJSON });
  }
}

// 2. 发送给所有有订阅的节点（不论订阅哪个会话）
sendToAllSubscribed(event, payload, sendEvent) {
  const payloadJSON = JSON.stringify(payload);
  for (const nodeId of nodeSubscriptions.keys()) {
    sendEvent({ nodeId, event, payloadJSON });
  }
}

// 3. 发送给所有在线节点（无论是否有订阅）
sendToAllConnected(event, payload, listConnected, sendEvent) {
  const payloadJSON = JSON.stringify(payload);
  for (const node of listConnected()) {
    sendEvent({ nodeId: node.nodeId, event, payloadJSON });
  }
}
```

`sendEvent` 参数是一个注入的回调函数，实际实现是通过 `NodeRegistry.sendEvent` 将消息写入 WebSocket。这种**依赖注入**设计使得订阅管理器完全不依赖 WebSocket 的具体实现，便于独立测试。

27.2.4 Gateway 方法处理（`src/gateway/server-methods/nodes.ts`）

方法概览

节点相关的 Gateway RPC 方法定义在 `server-methods/nodes.ts` 中：

方法	说明
`node.pair.request`	节点发起配对请求
`node.pair.list`	列出配对请求
`node.pair.approve`	批准配对
`node.pair.reject`	拒绝配对
`node.pair.verify`	验证节点令牌
`node.rename`	重命名节点
`node.list`	列出所有节点（配对 + 在线）
`node.describe`	查看单个节点详情
`node.invoke`	向节点发送命令
`node.invoke.result`	节点返回命令结果
`node.event`	节点上报事件

node.invoke 的完整流程

`node.invoke` 是最关键的方法——Agent 通过它向节点发送命令：

```
// src/gateway/server-methods/nodes.ts（简化）

"node.invoke": async ({ params, respond, context }) => {
  const { nodeId, command } = params;

  // 1. 查找节点
  const nodeSession = context.nodeRegistry.get(nodeId);
  if (!nodeSession) {
    respond(false, undefined, { code: "NOT_CONNECTED" });
    return;
  }

  // 2. 权限检查
  const cfg = loadConfig();
  const allowlist = resolveNodeCommandAllowlist(cfg, nodeSession);
  const allowed = isNodeCommandAllowed({
    command,
    declaredCommands: nodeSession.commands,
    allowlist,
  });
  if (!allowed.ok) {
    respond(false, undefined, { reason: allowed.reason });
    return;
  }

  // 3. 调用节点（通过 NodeRegistry.invoke）
  const res = await context.nodeRegistry.invoke({
    nodeId,
    command,
    params: params.params,
    timeoutMs: params.timeoutMs,
    idempotencyKey: params.idempotencyKey,
  });

  // 4. 返回结果
  if (!res.ok) {
    respond(false, undefined, { message: res.error?.message });
    return;
  }
  respond(true, { ok: true, nodeId, command, payload: res.payload }, undefined);
};
```

node.invoke.result 的安全校验

节点返回结果时，Gateway 会检查调用者身份：

```
"node.invoke.result": async ({ params, respond, context, client }) => {
  // 验证：返回结果的节点必须是被调用的节点
  const callerNodeId = client?.connect?.device?.id ?? client?.connect?.client?.id;
  if (callerNodeId && callerNodeId !== params.nodeId) {
    respond(false, undefined, { message: "nodeId mismatch" });
    return;
  }

  const ok = context.nodeRegistry.handleInvokeResult({
    id: params.id,
    nodeId: params.nodeId,
    ok: params.ok,
    payload: params.payload,
    payloadJSON: params.payloadJSON,
    error: params.error,
  });

  // 迟到的结果被静默忽略
  if (!ok) {
    context.logGateway.debug(`late invoke result ignored: id=${params.id}`);
    respond(true, { ok: true, ignored: true }, undefined);
    return;
  }
  respond(true, { ok: true }, undefined);
};
```

迟到的结果（invoke 已超时后才到达）不会产生错误——Gateway 返回 `{ ok: true, ignored: true }` 。这避免了网络延迟导致的无谓错误日志。

node.list 的融合视图

`node.list` 不仅返回当前在线的节点，还包括已配对但离线的节点，提供一个**融合视图**：

```
"node.list": async ({ params, respond, context }) => {
  // 获取已配对的节点列表
  const list = await listDevicePairing();
  const pairedById = new Map(/* 配对节点 → 基础信息 */);

  // 获取当前在线的节点列表
  const connected = context.nodeRegistry.listConnected();
  const connectedById = new Map(connected.map(n => [n.nodeId, n]));

  // 合并：取并集
  const nodeIds = new Set([...pairedById.keys(), ...connectedById.keys()]);
  const nodes = [...nodeIds].map(nodeId => {
    const paired = pairedById.get(nodeId);
    const live = connectedById.get(nodeId);
    return {
      nodeId,
      displayName: live?.displayName ?? paired?.displayName,
      // ... 在线信息优先，离线信息兜底
      paired: Boolean(paired),
      connected: Boolean(live),
    };
  });

  // 排序：在线优先，然后按名称
  nodes.sort((a, b) => {
    if (a.connected !== b.connected) return a.connected ? -1 : 1;
    return (a.displayName ?? a.nodeId).localeCompare(b.displayName ?? b.nodeId);
  });

  respond(true, { ts: Date.now(), nodes }, undefined);
};
```

本节小结

1. **NodeRegistry 使用双向索引**——`nodesById` 和 `nodesByConn` 两个 Map 支持按节点 ID 和连接 ID 双向查找，写入时同步维护一致性。
2. **invoke 实现请求-响应匹配**——通过 UUID 标识每个请求，将 `{ resolve, reject, timer }` 存入 `pendingInvokes`，节点返回结果后通过 ID 匹配完成 Promise。超时被视为正常失败而非异常。

3. **事件系统处理五类节点事件**——`voice.transcript`（语音转写）、`agent.request`（Agent 深链）、`chat.subscribe/unsubscribe`（订阅管理）、`exec.*`（执行状态）。
4. **订阅管理器维护多对多关系**——两个 Map 双向索引节点与会话的订阅关系，提供 `sendToSession`、`sendToAllSubscribed`、`sendToAllConnected` 三种分发方式。
5. **Gateway 方法实现完整的节点生命周期**——从配对（pair）到发现（list/describe）到调用（invoke）到事件（event），每个方法都包含参数校验和安全检查。
6. **迟到结果被优雅处理**——超时后到达的 invoke 结果不会产生错误，而是被静默忽略并记录调试日志。